

ASSIGNMENT NO. 11: Indexing and Query Optimization Queries

Objective:

The objective of this practical is to help students understand indexing and query optimization in MySQL. Students will learn the basic concept of indexes, identify suitable columns for indexing, create single-column and composite indexes, inspect indexes using SHOW INDEX, analyze query execution plans using EXPLAIN, and understand techniques for improving query efficiency.

1. Understanding Indexing Concepts

An **index** is a database structure that helps MySQL locate required records more efficiently. It works in a manner similar to the index of a book. Instead of reading every page, the reader uses the index to directly locate a topic.

Without an index, MySQL may need to examine many or all rows in a table to find matching records. With a suitable index, MySQL can locate matching rows more quickly. Indexes are especially useful for columns frequently used in:

Query Operation	Example
WHERE conditions	WHERE City = 'Mumbai'
Table joins	ON Customer.CustomerID = Orders.CustomerID
Sorting	ORDER BY OrderDate
Grouping	GROUP BY City
Range searches	OrderDate BETWEEN '2026-01-01' AND '2026-03-31'
Exact searches	WHERE Email = 'rahul@gmail.com'

MySQL documentation explains that indexes act like pointers to table rows and can improve searches involving conditions such as =, >, BETWEEN, and IN. However, unnecessary indexes use storage and increase the work required during INSERT, UPDATE, and DELETE operations.

Simple Comparison

Without Index	With Index
MySQL may scan many table rows	MySQL can directly search indexed values
Query may become slow on large tables	Query may execute faster
Less storage is required	Additional storage is required
Faster data insertion	Insert and update operations may require extra index maintenance

Example

Suppose an OrderData table contains 10 lakh records and the following query is executed:

```
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

If City is not indexed, MySQL may examine a large number of rows.

If an index is created on City, MySQL may use the index to locate matching records more efficiently.

```
CREATE INDEX idx_city
ON OrderData(City);
```

Common Types of Indexes

Index Type	Purpose
Primary index	Automatically associated with a primary key
Unique index	Prevents duplicate values
Single-column index	Created on one column
Composite index	Created on two or more columns
Full-text index	Used for searching words in text columns
Spatial index	Used for spatial or geographical data

Important Point for InnoDB

In an InnoDB table, the primary key is generally used as the clustered index. Secondary indexes also contain the primary-key value needed to locate the complete row.

2. Creating a Sample Database and Table

For this practical, we will create a database named SalesDB and a table named OrderData.

OrderData Table Structure

Column Name	Data Type	Constraint	Description
OrderID	INT	PRIMARY KEY	Unique order number
CustomerName	VARCHAR(50)	NOT NULL	Name of the customer
Email	VARCHAR(80)	NOT NULL	Customer email address
City	VARCHAR(30)	NOT NULL	Customer city
ProductCategory	VARCHAR(30)	NOT NULL	Product category
OrderDate	DATE	NOT NULL	Date of order
OrderAmount	DECIMAL(10,2)	NOT NULL	Total order value
OrderStatus	VARCHAR(20)	NOT NULL	Current order status

MySQL Code to Create Database and Table

```
CREATE DATABASE SalesDB;
```

```
USE SalesDB;
```

```
CREATE TABLE OrderData (
    OrderID INT PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL,
    Email VARCHAR(80) NOT NULL,
    City VARCHAR(30) NOT NULL,
    ProductCategory VARCHAR(30) NOT NULL,
    OrderDate DATE NOT NULL,
    OrderAmount DECIMAL(10,2) NOT NULL,
    OrderStatus VARCHAR(20) NOT NULL
);
```

Insert Sample Records

```
INSERT INTO OrderData
```

```
(
    OrderID,
    CustomerName,
    Email,
    City,
    ProductCategory,
    OrderDate,
    OrderAmount,
    OrderStatus
)
VALUES
(1, 'Rahul Sharma', 'rahul@gmail.com', 'Mumbai',
 'Electronics', '2026-01-05', 55000.00, 'Delivered'),

(2, 'Priya Patil', 'priya@gmail.com', 'Pune',
 'Furniture', '2026-01-08', 32000.00, 'Delivered'),

(3, 'Aman Verma', 'aman@gmail.com', 'Nashik',
 'Clothing', '2026-01-12', 8500.00, 'Pending'),

(4, 'Sneha Joshi', 'sneha@gmail.com', 'Mumbai',
 'Electronics', '2026-01-20', 72000.00, 'Delivered'),

(5, 'Karan Mehta', 'karan@gmail.com', 'Thane',
 'Furniture', '2026-02-03', 28000.00, 'Cancelled'),

(6, 'Neha Shah', 'neha@gmail.com', 'Pune',
 'Electronics', '2026-02-08', 48000.00, 'Delivered'),

(7, 'Rohan Desai', 'rohan@gmail.com', 'Mumbai',
 'Clothing', '2026-02-15', 12000.00, 'Pending'),

(8, 'Meera Kulkarni', 'meera@gmail.com', 'Nashik',
 'Furniture', '2026-02-22', 36000.00, 'Delivered'),

(9, 'Arjun Nair', 'arjun@gmail.com', 'Thane',
 'Electronics', '2026-03-02', 64000.00, 'Shipped'),

(10, 'Divya More', 'divya@gmail.com', 'Mumbai',
 'Furniture', '2026-03-07', 41000.00, 'Delivered'),
```

```
(11, 'Isha Singh', 'isha@gmail.com', 'Pune',
'Clothing', '2026-03-12', 15000.00, 'Shipped'),

(12, 'Om Jadhav', 'om@gmail.com', 'Mumbai',
'Electronics', '2026-03-18', 59000.00, 'Pending');
```

OrderData Table

OrderID	Customer Name	Email	City	Product Category	Order Date	Order Amount	Order Status
1	Rahul Sharma	rahul@gmail.com	Mumbai	Electronics	2026-01-05	55000.00	Delivered
2	Priya Patil	priya@gmail.com	Pune	Furniture	2026-01-08	32000.00	Delivered
3	Aman Verma	aman@gmail.com	Nashik	Clothing	2026-01-12	8500.00	Pending
4	Sneha Joshi	sneha@gmail.com	Mumbai	Electronics	2026-01-20	72000.00	Delivered
5	Karan Mehta	karan@gmail.com	Thane	Furniture	2026-02-03	28000.00	Cancelled
6	Neha Shah	neha@gmail.com	Pune	Electronics	2026-02-08	48000.00	Delivered
7	Rohan Desai	rohan@gmail.com	Mumbai	Clothing	2026-02-15	12000.00	Pending
8	Meera Kulkarni	meera@gmail.com	Nashik	Furniture	2026-02-22	36000.00	Delivered
9	Arjun Nair	arjun@gmail.com	Thane	Electronics	2026-03-02	64000.00	Shipped
10	Divya More	divya@gmail.com	Mumbai	Furniture	2026-03-07	41000.00	Delivered
11	Isha Singh	isha@gmail.com	Pune	Clothing	2026-03-12	15000.00	Shipped
12	Om Jadhav	om@gmail.com	Mumbai	Electronics	2026-03-18	59000.00	Pending

3. Creating and Managing Indexes

MySQL provides the CREATE INDEX command to create an index on an existing table.

General Syntax

```
CREATE INDEX index_name ON
table_name(column_name);
```

Creating a Single-Column Index

A single-column index is created on one table column.

Example: Create Index on City

```
CREATE INDEX idx_city
ON OrderData(City);
```

This index can support queries such as:

```
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

Creating an Index on OrderDate

```
CREATE INDEX idx_order_date
ON OrderData(OrderDate);
```

This index may support date-based queries such as:

```
SELECT *
FROM OrderData
WHERE OrderDate BETWEEN '2026-01-01' AND '2026-02-28';
```

Creating a Unique Index

A unique index does not allow duplicate values in the indexed column.

```
CREATE UNIQUE INDEX idx_email
ON OrderData(Email);
```

After creating the unique index, MySQL will reject an attempt to insert a duplicate email address.

Example of Invalid Duplicate Value

```
INSERT INTO OrderData
VALUES
(
    13,
    'New Customer',
    'rahul@gmail.com',
    'Mumbai',
    'Electronics',
    '2026-03-20',
    25000.00,
    'Pending'
);
```

The record will not be inserted because rahul@gmail.com already exists.

Creating a Composite Index

A composite index is created on two or more columns.

```
CREATE INDEX idx_city_order_date
ON OrderData(City, OrderDate);
```

This index is designed for queries that search by City and OrderDate.

```
SELECT *
FROM OrderData
WHERE City = 'Mumbai'
AND OrderDate BETWEEN '2026-01-01' AND '2026-03-31';
```

Composite Index Column Order

For the following index:

```
CREATE INDEX idx_city_order_date
ON OrderData(City, OrderDate);
```

The index is generally most directly useful for these conditions:

Query Condition	Index Use
WHERE City = 'Mumbai'	Can use the index
WHERE City = 'Mumbai' AND OrderDate >= '2026-01-01'	Can use the index
WHERE OrderDate >= '2026-01-01' without City	May not use the complete composite index efficiently

Viewing Existing Indexes

The SHOW INDEX command displays information about indexes defined on a table.

```
SHOW INDEX FROM OrderData;
```

Important SHOW INDEX Columns

Output Column	Meaning
Key_name	Name of the index
Column_name	Column included in the index
Non_unique	0 for unique index and 1 for non-unique index
Seq_in_index	Position of the column in a composite index
Cardinality	Estimated number of distinct index values
Index_type	Type of index, such as BTREE
Visible	Indicates whether the index is visible to the optimizer

Removing an Index

An unnecessary index can be deleted using DROP INDEX.

```
DROP INDEX idx_order_date
ON OrderData;
```

Index Management Summary

Requirement	MySQL Command
-------------	---------------

Create normal index	CREATE INDEX
Create unique index	CREATE UNIQUE INDEX
Create composite index	CREATE INDEX ... (column1, column2)
Display indexes	SHOW INDEX FROM table_name
Remove index	DROP INDEX index_name ON table_name

4. Choosing Suitable Columns for Indexing

Indexes should be created according to actual query requirements. Creating an index on every column is not a good optimization strategy.

Suitable Columns for Indexing

Column Type or Usage	Example
Columns frequently used in WHERE	City, Email
Columns used in joins	CustomerID, ProductID
Columns used for date ranges	OrderDate
Columns used for sorting	OrderAmount
Columns containing mostly unique values	Email, EmployeeCode
Foreign-key columns	CourseID, DepartmentID

Columns That May Not Benefit Much

Column Situation	Example	Reason
Very small table	Table with 10 rows	Full scan may already be inexpensive
Low-cardinality column	Gender with two values	Many rows may match each value
Frequently updated column	Rapidly changing status field	Index maintenance adds overhead
Column rarely used in queries	Remarks	Index may never be used
Very large text field	Description	Normal index may be large or unsuitable

Selectivity

Selectivity describes how effectively a column separates records.

High-Selectivity Columns

Column	Example
Email	Mostly unique for each customer
OrderID	Unique for each order
EmployeeCode	Unique employee value

Low-Selectivity Columns

Column	Example Values
OrderStatus	Pending, Shipped, Delivered
Gender	Male, Female
ActiveStatus	Yes, No

A high-selectivity index is often more useful because it can reduce the number of matching rows significantly.

Example Comparison

```
SELECT *
FROM OrderData
WHERE Email = 'rahul@gmail.com';
```

The email condition is highly selective because it normally returns one record.

```
SELECT *
FROM OrderData
WHERE OrderStatus = 'Delivered';
```

The order-status condition may return many records and therefore may be less selective.

Benefits and Costs of Indexes

Benefits	Costs
Faster record searching	Additional disk storage
Faster joins	Slower insertion
Faster range filtering	Slower updates to indexed columns
May reduce sorting work	Slower deletion
Supports uniqueness	Additional maintenance

MySQL recommends balancing faster searches against the storage and data-modification cost of maintaining unnecessary indexes.

5. Analyzing Query Performance Using EXPLAIN

The EXPLAIN statement displays the execution plan selected by the MySQL optimizer. It helps identify how MySQL intends to access tables, which indexes may be considered, which index is selected, and approximately how many rows may be examined.

Basic Syntax

```
EXPLAIN
SELECT column_names
FROM table_name
WHERE condition;
```

Example 1: Analyze a City Search

```
EXPLAIN
```

```

SELECT *
FROM OrderData
WHERE City = 'Mumbai';

```

Important EXPLAIN Columns

Column	Meaning
id	Identifier of the query step
select_type	Type of SELECT operation
table	Table accessed by the query
type	Access method used by MySQL
possible_keys	Indexes MySQL may be able to use
key	Index actually selected
key_len	Length of the selected index
ref	Value or column compared with the index
rows	Estimated number of rows examined
filtered	Estimated percentage of rows passing the condition
Extra	Additional execution information

Common Values in the type Column

Access Type	General Meaning
const	MySQL expects one matching row using a unique or primary key
Access Type	General Meaning
eq_ref	One matching row is read for each row from another table
ref	Index lookup may return multiple matching rows
range	Index is used for a range of values
index	MySQL scans the index
ALL	MySQL performs a full table scan

Simplified Interpretation

EXPLAIN Observation	Possible Meaning
type = ALL	Full table scan may be occurring
possible_keys = NULL	No suitable index was identified
key = NULL	No index was selected
key = idx_city	MySQL selected the City index
Lower estimated rows	Fewer rows may need examination
Using where	Rows are filtered using a WHERE condition
Using filesort	Additional sorting may be required
Using temporary	MySQL may create a temporary result

Comparing a Query Before and After Indexing Step 1: Remove the City Index, if it exists

```
DROP INDEX idx_city
ON OrderData;
```

Step 2: Examine the Query

```
EXPLAIN
SELECT *
FROM OrderData WHERE
City = 'Mumbai';
```

A possible plan may show:

Item	Possible Observation
type	ALL
possible_keys	NULL
key	NULL
rows	Many or all table rows

Step 3: Create the Index

```
CREATE INDEX idx_city
ON OrderData (City);
```

Step 4: Examine the Query Again

```
EXPLAIN
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

A possible plan on a sufficiently large and selective dataset may show:

Item	Possible Observation
type	ref
possible_keys	idx_city
Item	Possible Observation
key	idx_city
rows	Fewer estimated rows

Important Note

The optimizer may still choose a full table scan for a very small table because scanning a few rows may be less expensive than using an index. Therefore, students should not expect exactly the same EXPLAIN output on every computer or dataset.

EXPLAIN with a Range Query

```
EXPLAIN
SELECT *
FROM OrderData
```

```
WHERE OrderDate BETWEEN '2026-01-01' AND '2026-02-28';
```

If an index exists on OrderDate, the access type may appear as range.

EXPLAIN with Composite Index

```
EXPLAIN
SELECT *
FROM OrderData
WHERE City = 'Mumbai'
AND OrderDate BETWEEN '2026-01-01' AND '2026-03-31';
```

This query may use: idx_city_order_date

Optional EXPLAIN Formats

MySQL supports different EXPLAIN output formats, including traditional tabular output, JSON, and tree output.

EXPLAIN FORMAT=JSON

```
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

EXPLAIN FORMAT=TREE

```
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

6. Improving Query Efficiency

Query optimization means improving SQL statements and database structures so that MySQL can produce the required result using fewer resources.

Technique 1: Select Only Required Columns

Avoid retrieving all columns when only a few are required.

Less Efficient Query

```
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

Improved Query

```
SELECT OrderID, CustomerName, OrderDate, OrderAmount
FROM OrderData
WHERE City = 'Mumbai';
```

Technique 2: Index Frequently Searched Columns

```
CREATE INDEX idx_city
ON OrderData(City);
```

Technique 3: Use Composite Indexes for Combined Conditions

```
CREATE INDEX idx_city_order_date
ON OrderData(City, OrderDate);
```

This index may help the following query:

```
SELECT OrderID, CustomerName, OrderDate, OrderAmount
FROM OrderData
WHERE City = 'Mumbai'
AND OrderDate >= '2026-01-01';
```

Technique 4: Avoid Functions on Indexed Columns

The following query applies a function to the indexed column:

```
SELECT *
FROM OrderData
WHERE YEAR(OrderDate) = 2026;
```

A range condition may be more index-friendly:

```
SELECT *
FROM OrderData
WHERE OrderDate >= '2026-01-01'
AND OrderDate < '2027-01-01';
```

Technique 5: Avoid Starting LIKE with a Wildcard

The following pattern begins with %:

```
SELECT *
FROM OrderData
WHERE CustomerName LIKE '%Sharma';
```

A search with a known starting value may be more suitable for a normal B-tree index:

```
SELECT *
FROM OrderData
WHERE CustomerName LIKE 'Rahul%';
```

Technique 6: Use Appropriate Data Types

Poor Choice	Better Choice
Store date as VARCHAR	Use DATE
Store amount as text	Use DECIMAL
Use oversized text columns	Use suitable length
Store numeric IDs as text	Use INT

Technique 7: Avoid Unnecessary Indexes

Before creating an index, ask:

Question	Purpose
Is the column frequently searched?	Confirms practical use
Is it used in a join?	May improve join performance
Question	Purpose
Does it have enough distinct values?	Checks selectivity
Is the table large enough?	Small tables may not benefit
Is a similar composite index already present?	Avoids duplication

Technique 8: Use EXPLAIN Before and After Optimization**EXPLAIN**

```
SELECT OrderID, CustomerName, OrderAmount
FROM OrderData
WHERE City = 'Mumbai';
```

After creating the index, run the same EXPLAIN query again and compare:

Item to Compare
Access type
Possible indexes
Selected index
Estimated rows
Extra information

Query Optimization Summary

Technique	Benefit
Select required columns	Reduces transferred and processed data
Index WHERE columns	Improves record searching
Index join columns	May improve table joins
Use composite indexes carefully	Supports multiple related conditions
Use suitable data types	Reduces storage and comparison work
Avoid unnecessary functions	May allow index use
Check with EXPLAIN	Verifies optimizer decisions
Remove unused indexes	Reduces storage and DML overhead

7. Practical Query Optimization Scenarios

Indexes and EXPLAIN are used in practical systems such as e-commerce, banking, education, hospital management, and employee-management applications.

Scenario-Based Examples

Scenario	Requirement	Suggested Index
Customer search	Find customer using email	Unique index on Email
City report	Display orders from one city	Index on City
Date report	Display orders within a date range	Index on OrderDate
City and date report	Search using city and date	Composite index on City and OrderDate
Product analysis	Search by product category	Index on ProductCategory

Order lookup	Find one order using OrderID	Primary-key index
--------------	------------------------------	-------------------

Scenario 1: Customer Search by Email

```
CREATE UNIQUE INDEX idx_email
ON OrderData(Email);
```

EXPLAIN

```
SELECT OrderID, CustomerName, City
FROM OrderData
WHERE Email = 'rahul@gmail.com';
```

Expected Concept

Observation	Explanation
Unique lookup	Only one email should match
Selected index	idx_email may be selected
Small row estimate	MySQL expects very few matching rows

Scenario 2: Date-Wise Order Report

```
CREATE INDEX idx_order_date
ON OrderData(OrderDate);
```

```
SELECT OrderID, CustomerName, OrderDate, OrderAmount
FROM OrderData
WHERE OrderDate BETWEEN '2026-02-01' AND '2026-02-28'
ORDER BY OrderDate;
```

Output

OrderID	CustomerName	OrderDate	OrderAmount
5	Karan Mehta	2026-02-03	28000.00
6	Neha Shah	2026-02-08	48000.00
7	Rohan Desai	2026-02-15	12000.00
8	Meera Kulkarni	2026-02-22	36000.00

Scenario 3: Mumbai Orders Within a Date Range

```
CREATE INDEX idx_city_order_date
ON OrderData(City, OrderDate);
SELECT
    OrderID,
    CustomerName,
    City,
    OrderDate,
    OrderAmount
FROM OrderData
WHERE City = 'Mumbai'
AND OrderDate BETWEEN '2026-02-01' AND '2026-03-31'
ORDER BY OrderDate;
```

Output

OrderID	CustomerName	City	OrderDate	OrderAmount
7	Rohan Desai	Mumbai	2026-02-15	12000.00
10	Divya More	Mumbai	2026-03-07	41000.00
12	Om Jadhav	Mumbai	2026-03-18	59000.00

Scenario 4: Compare Query Plans

EXPLAIN

SELECT

```
OrderID,
CustomerName,
City,
OrderDate,
OrderAmount
```

FROM OrderData

WHERE City = 'Mumbai'

AND OrderDate **BETWEEN** '2026-02-01' **AND** '2026-03-31';

Students should observe:

EXPLAIN Field	What to Check
possible_keys	Whether idx_city_order_date is listed
key	Whether MySQL selected the composite index
type	Whether access is ref or range rather than ALL
rows	Estimated rows examined
Extra	Additional filtering or sorting information

Final Summary Table

Concept	MySQL Command	Purpose
Create single index	CREATE INDEX	Indexes one column
Create unique index	CREATE UNIQUE INDEX	Prevents duplicate indexed values
Create composite index	CREATE INDEX ... (col1, col2)	Indexes multiple columns
Display indexes	SHOW INDEX	Shows table index information
Remove index	DROP INDEX	Deletes an unnecessary index
Analyze query	EXPLAIN	Displays query execution plan
Analyze detailed plan	EXPLAIN FORMAT=JSON	Displays detailed JSON plan
Verify selected index	Check key column	Shows index chosen by optimizer
Identify full scan	Check type = ALL	Indicates table scan
Improve query	Indexing and query rewriting	Reduces unnecessary processing

Questions

Q1) Create a MySQL database named SalesDB and a table named OrderData.

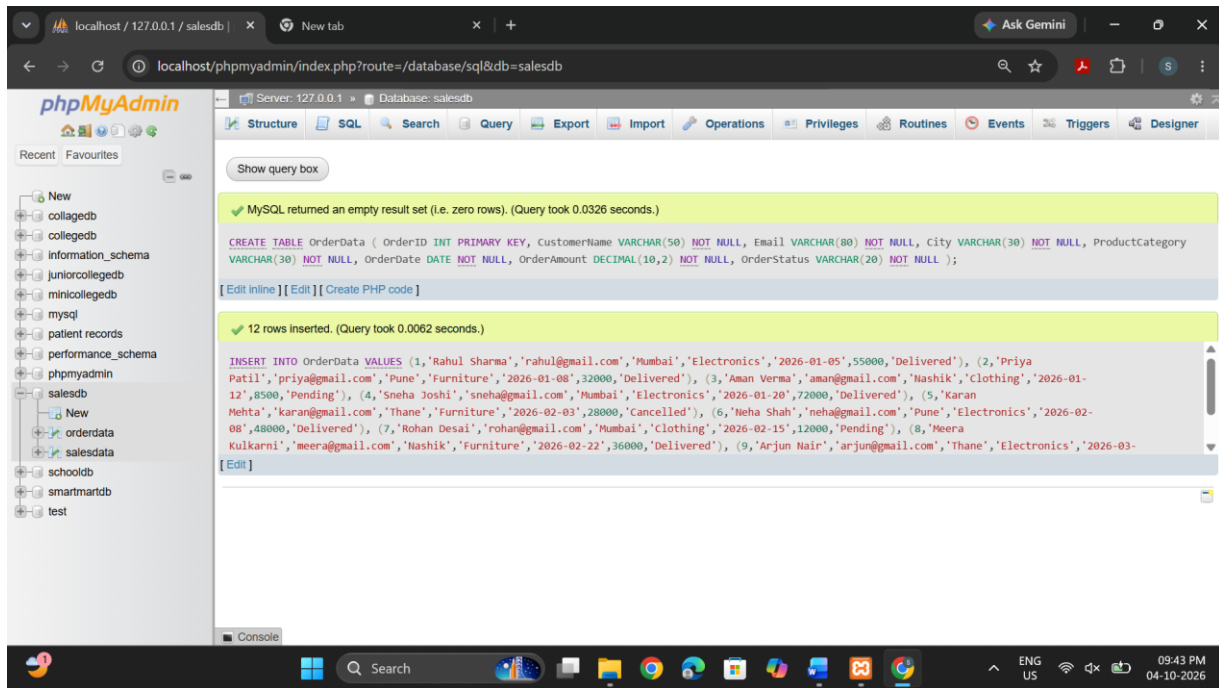
OrderData Table

Column Name	Data Type	Constraint
OrderID	INT	PRIMARY KEY
CustomerName	VARCHAR(50)	NOT NULL
Email	VARCHAR(80)	NOT NULL
City	VARCHAR(30)	NOT NULL
ProductCategory	VARCHAR(30)	NOT NULL
OrderDate	DATE	NOT NULL
OrderAmount	DECIMAL(10,2)	NOT NULL
OrderStatus	VARCHAR(20)	NOT NULL

Insert at least 12 records into the table.

```
CREATE TABLE OrderData (
  OrderID INT PRIMARY KEY,
  CustomerName VARCHAR(50) NOT NULL,
  Email VARCHAR(80) NOT NULL,
  City VARCHAR(30) NOT NULL,
  ProductCategory VARCHAR(30) NOT NULL,
  OrderDate DATE NOT NULL,
  OrderAmount DECIMAL(10,2) NOT NULL,
  OrderStatus VARCHAR(20) NOT NULL
);
```

```
INSERT INTO OrderData VALUES
(1,'Rahul Sharma','rahul@gmail.com','Mumbai','Electronics','2026-01-05',55000,'Delivered'),
(2,'Priya Patil','priya@gmail.com','Pune','Furniture','2026-01-08',32000,'Delivered'),
(3,'Aman Verma','aman@gmail.com','Nashik','Clothing','2026-01-12',8500,'Pending'),
(4,'Sneha Joshi','sneha@gmail.com','Mumbai','Electronics','2026-01-20',72000,'Delivered'),
(5,'Karan Mehta','karan@gmail.com','Thane','Furniture','2026-02-03',28000,'Cancelled'),
(6,'Neha Shah','neha@gmail.com','Pune','Electronics','2026-02-08',48000,'Delivered'),
(7,'Rohan Desai','rohan@gmail.com','Mumbai','Clothing','2026-02-15',12000,'Pending'),
(8,'Meera Kulkarni','meera@gmail.com','Nashik','Furniture','2026-02-22',36000,'Delivered'),
(9,'Arjun Nair','arjun@gmail.com','Thane','Electronics','2026-03-02',64000,'Shipped'),
(10,'Divya More','divya@gmail.com','Mumbai','Furniture','2026-03-07',41000,'Delivered'),
(11,'Isha Singh','isha@gmail.com','Pune','Clothing','2026-03-12',15000,'Shipped'),
(12,'Om Jadhav','om@gmail.com','Mumbai','Electronics','2026-03-18',59000,'Pending');
```



Q2) Create the following MySQL indexes on the OrderData table:

CREATE INDEX idx_city ON OrderData(City);

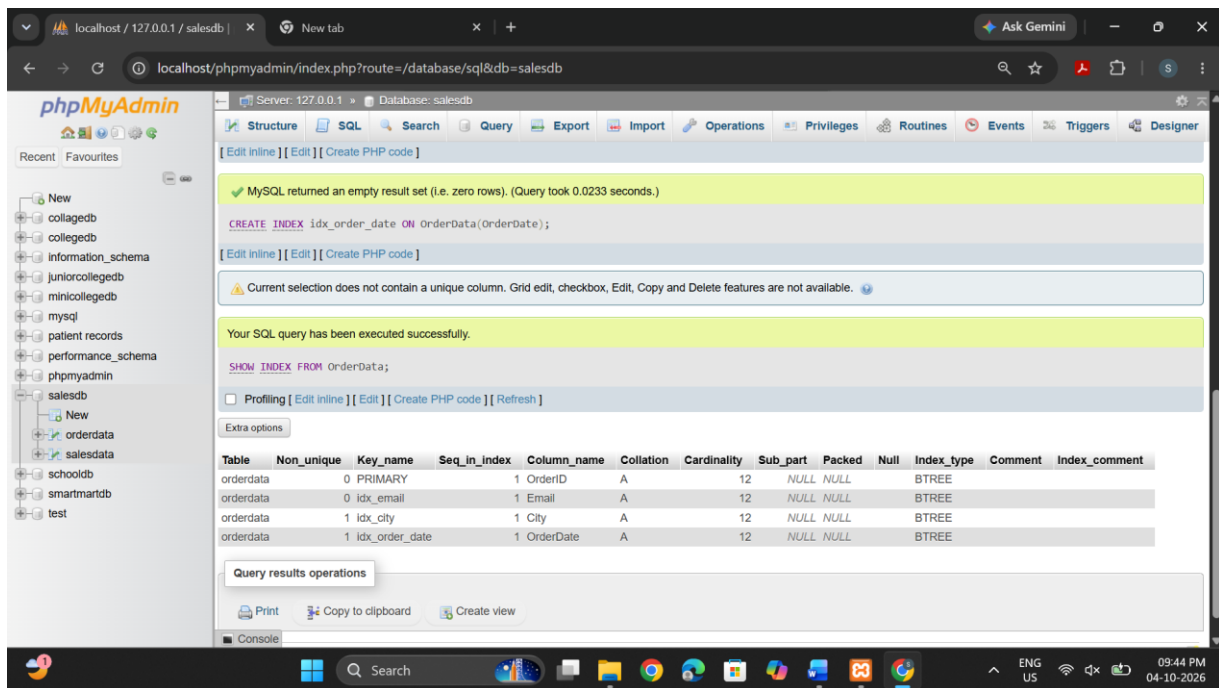
CREATE UNIQUE INDEX idx_email ON OrderData(Email);

CREATE INDEX idx_order_date ON OrderData(OrderDate);

SHOW INDEX FROM OrderData;

Index Name	Column	Index Type
idx_city	City	Normal index
idx_email	Email	Unique index
idx_order_date	OrderDate	Normal index

Display all indexes using SHOW INDEX.



Q3) Create a composite index named idx_city_order_date.

```
CREATE INDEX idx_city_order_date
```

```
ON OrderData(City, OrderDate);
```

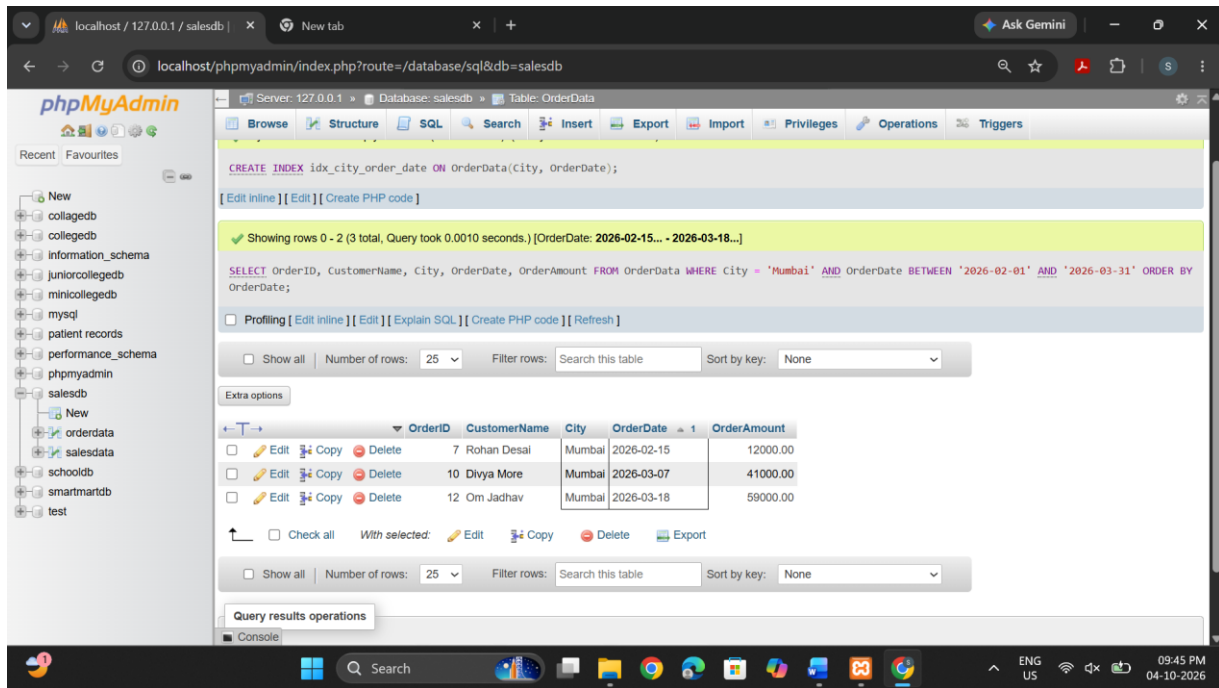
```
SELECT OrderID, CustomerName, City, OrderDate, OrderAmount
```

```
FROM OrderData
```

```
WHERE City = 'Mumbai'
```

```
AND OrderDate BETWEEN '2026-02-01' AND '2026-03-31'
```

```
ORDER BY OrderDate;
```



The index should contain:

Index Position	Column
1	City
2	OrderDate

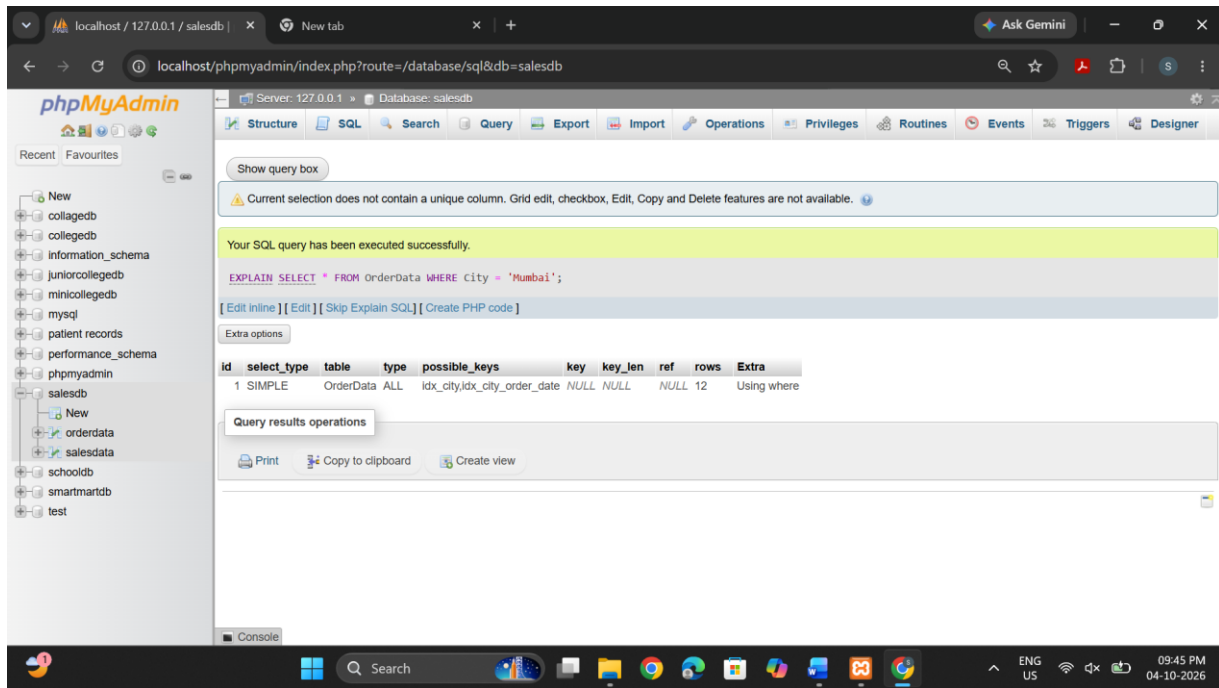
Write a query that searches for Mumbai orders within a specified date range.

Q4) Write a MySQL EXPLAIN query to analyze the following search:

SELECT *

FROM OrderData

WHERE City = 'Mumbai';



Record and explain the following output columns:

EXPLAIN Column
type
possible_keys
key
rows
Extra

Q5) Compare query performance before and after creating an index on City.

-- Step 1

DROP INDEX idx_city ON OrderData;

-- Step 2

EXPLAIN

SELECT * FROM OrderData

WHERE City = 'Mumbai';

-- Step 3

CREATE INDEX idx_city ON OrderData(City);

-- Step 4**EXPLAIN****SELECT * FROM OrderData****WHERE City = 'Mumbai';**

The first screenshot shows the phpMyAdmin interface with the 'salesdb' database selected. The 'Query' tab is active, and the query box contains the following SQL statement:

```
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0197 seconds.)
DROP INDEX idx_city ON OrderData;
```

The second screenshot shows the same interface after executing the query. A message indicates that the current selection does not contain a unique column. The 'Query' tab shows the successful execution of the following SQL statement:

```
EXPLAIN SELECT * FROM OrderData WHERE City = 'Mumbai';
```

The 'Extra options' section is expanded, showing the following table:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	OrderData	ALL	idx_city_order_date	NULL	NULL	NULL	12	Using where

The 'Query results operations' section is also visible, showing options to Print, Copy to clipboard, and Create view.

The top screenshot shows the phpMyAdmin interface with the 'salesdb' database selected. The 'SQL' tab is active, and a query has been executed: `CREATE INDEX idx_city ON OrderData(City);`. The message indicates: "MySQL returned an empty result set (i.e. zero rows). (Query took 0.0202 seconds.)".

The bottom screenshot shows the same interface after running an `EXPLAIN` query: `EXPLAIN SELECT * FROM OrderData WHERE City = 'Mumbai';`. A warning message states: "Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available." Below this, a message says: "Your SQL query has been executed successfully." The 'EXPLAIN' results are shown in a table:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	OrderData	ALL	idx_city_order_date,idx_city	NULL	NULL	NULL	12	Using where

Below the table, the 'Query results operations' section shows options: Print, Copy to clipboard, and Create view.

Perform the following steps:

Step	Task
1	Drop idx_city, if it exists
2	Run EXPLAIN on the City search
3	Create idx_city
4	Run the same EXPLAIN query again
5	Compare the selected index and estimated rows

Q6) Rewrite the following query in a more index-friendly form:

```
SELECT *
FROM OrderData
WHERE YEAR(OrderDate) = 2026;
```

```
SELECT OrderID, CustomerName,
OrderDate, OrderAmount

FROM OrderData

WHERE OrderDate >= '2026-01-01'

AND OrderDate < '2027-01-01';
```

EXPLAIN

```
SELECT OrderID, CustomerName,
OrderDate, OrderAmount

FROM OrderData

WHERE OrderDate >= '2026-01-01'

AND OrderDate < '2027-01-01';
```

The screenshot shows the phpMyAdmin interface for a database named 'salesdb'. The 'OrderData' table is selected, displaying 12 rows of data. The columns are OrderID, CustomerName, OrderDate, and OrderAmount. The data is as follows:

OrderID	CustomerName	OrderDate	OrderAmount
1	Rahul Sharma	2026-01-05	55000.00
2	Priya Patil	2026-01-08	32000.00
3	Aman Verma	2026-01-12	8500.00
4	Sneha Joshi	2026-01-20	72000.00
5	Karan Mehta	2026-02-03	28000.00
6	Neha Shah	2026-02-08	48000.00
7	Rohan Desai	2026-02-15	12000.00
8	Meera Kulkarni	2026-02-22	36000.00
9	Arjun Nair	2026-03-02	64000.00
10	Divya More	2026-03-07	41000.00
11	Isha Singh	2026-03-12	15000.00
12	Om Jadhav	2026-03-18	59000.00

The improved query should:

Requirement

Avoid applying a function to OrderDate

Use a start date
Use an end date
Select only OrderID, CustomerName, OrderDate, and OrderAmount
Be analyzed using EXPLAIN

Assignment Evaluation

- 0: Not Done []
1: Incomplete []
2: Late Complete []
3: Needs Improvement []
4: Complete []
5: Well Done []

Signature of Instructor: _____

